

Coding & Solution

Date	06 July 2026
Team ID	xxxxxx
Project Name	Smart Lender
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/puttasiri1/Smart-Lender-Loan-Approval-Prediction
Programming Language(s)	Python 3.11
Framework(s) / Libraries Used	Flask (Web Framework), Scikit-learn & XGBoost (ML), Pandas & NumPy (Data Processing), Matplotlib & Seaborn (Visualization)
Key Features Implemented	1. Data preprocessing and encoding of categorical applicant features. 2. Exploratory Data Analysis (EDA) with correlation heatmaps and distribution plots. 3. Machine learning model training and comparison (Decision Tree, Random Forest, KNN, XGBoost) to predict loan approval; XGBoost selected as best model (94.7% training / 81.1% testing accuracy). 4. Interactive Flask web application for applicant input and real-time approval prediction. 5. Result display with approval/rejection outcome and risk classification.
Pending / Incomplete Features	1. Final cloud deployment on IBM Cloud (currently verified on a hosted test server). 2. Batch prediction via CSV upload for financial analysts. 3. Prediction history storage and export for credit officers.
Setup / Run Instructions	1. Clone the repository: <code>git clone <repo-url></code> 2. Navigate to the project directory: <code>cd smart-lender-loan-prediction</code> 3. Install dependencies: <code>pip install -r requirements.txt</code> 4. Run the Flask application: <code>python app.py</code> 5. Open the browser and go to <code>http://127.0.0.1:5000/</code> to access the UI.

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

- **Code Standards:** The codebase strictly follows PEP-8 guidelines for Python, ensuring consistent indentation, spacing, and naming conventions (e.g., snake_case for variables/functions).
- **Modularity:** The project is divided into logical modules such as data_preprocessing.py, model_training.py, and app.py (Flask routes) to ensure high reusability and maintainability.
- **Error Handling:** Form inputs in the Flask application are validated to ensure applicants enter realistic values for income, loan amount, and loan term. Invalid inputs trigger user-friendly error messages.
- **Team Collaboration:** The team utilized Git for version control, with each member working on separate branches for tasks such as EDA, model tuning, and frontend integration before merging into the main branch.
- **Future Scope:** The current model can be further enhanced by integrating additional applicant features (e.g., transaction history) or adding an explainability layer (e.g., SHAP values) to justify individual approval/rejection decisions to credit officers.